

AI Practice Exams: Answer Keys

Practice Exam 1: Answer Key

Part I. Identification

1. Turing Test Approach (acting humanly)
2. Dartmouth College AI Workshop
3. Satisficing
4. R1
5. Law 4: Search compensates for lack of knowledge
6. Architecture + program
7. Performance measure, Environment, Actuators, Sensors
8. Problem generator
9. Semi-dynamic
10. Fringe (frontier)
11. Iterative deepening search
12. h_2 dominates h_1
13. Path max equation
14. Horizon effect
15. Selection, expansion, simulation, back-propagation
16. Solution
17. Pocket algorithm
18. F_1 -score
19. Synapse
20. Local gradient

Part II. True or False

1. FALSE. The emphasis is on action/behavior.
2. FALSE. Rationality maximizes expected performance; perfection maximizes actual performance.
3. TRUE.
4. FALSE. Only when path cost is a nondecreasing function of depth (for example, identical step costs).
5. TRUE. It stores the current path plus unexpanded siblings.
6. FALSE. Space is $O(bd)$; time is $O(b^d)$.
7. FALSE. Greedy search is neither complete nor optimal.
8. FALSE. Consistency implies admissibility, not the reverse.
9. TRUE. $f(n) = g(n)$.
10. FALSE. It returns the same value as minimax; it only skips branches that cannot affect the result.
11. TRUE.
12. FALSE. Arc consistency detects failure earlier; forward checking misses conflicts between two unassigned variables.
13. TRUE.
14. FALSE. Only when the classes are linearly separable (and ρ_t is properly chosen).
15. TRUE. Both equal $\frac{TP}{TP + FN}$, the true positive rate.
16. FALSE. Accuracy is useless for highly unbalanced data.

17. TRUE.
18. FALSE. It can separate unions of polyhedral regions but not any arbitrary union.
19. FALSE. That is pattern mode. Batch mode updates after all examples, at the end of each epoch.
20. FALSE. It converges toward some local minimum, not necessarily the global one.

Part III. Proving and Derivation

1. A* with an admissible heuristic is optimal (5 points).

Let G be an optimal goal with path cost C^* . Suppose, for contradiction, that A* selects a suboptimal goal G_2 for expansion, so

$$g(G_2) > C^*. \quad (1)$$

At that moment some node n on an optimal path to G is on the frontier. Since h is admissible, $h(n) \leq h^*(n)$, so

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*.$$

A* selected G_2 over n , so $f(G_2) \leq f(n) \leq C^*$. Since G_2 is a goal, $h(G_2) = 0$ and $f(G_2) = g(G_2)$. Hence

$$g(G_2) \leq C^*, \quad (2)$$

which contradicts (1). Therefore A* never selects a suboptimal goal. ■

Marking: assumption (1), 1 pt; frontier node on the optimal path, 1 pt; $f(n) \leq C^*$ from admissibility, 1 pt; selection rule and $h(G_2) = 0$, 1 pt; contradiction, 1 pt.

2. Manhattan distance (5 points).

(a) *Admissible.* Each move shifts exactly one tile by one square, so it reduces the sum of Manhattan distances by at most 1. Reaching the goal, where the sum is 0, therefore needs at least $h_2(n)$ moves: $h_2(n) \leq h^*(n)$.

(b) *Consistent.* Let n' be a successor of n . One move changes the distance of exactly one tile by ± 1 , so $h_2(n) - h_2(n') \leq 1 = c(n, n')$, that is

$$h_2(n) \leq c(n, n') + h_2(n').$$

Also $h_2(\text{goal}) = 0$. ■

3. Full derivation of the backpropagation algorithm (10 points).

Starting point. Gradient descent on the instantaneous error:

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)}.$$

(a) **Output neuron (4 points).** w_{ji} affects $\mathcal{E}$ only through v_j , then y_j , then e_j . By the chain rule,

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)}.$$

From the definitions:

$$\begin{aligned} \mathcal{E}(n) &= \frac{1}{2} \sum_{j \in C} e_j^2(n) & \Rightarrow & \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} = e_j(n), \\ e_j(n) &= d_j(n) - y_j(n) & \Rightarrow & \frac{\partial e_j(n)}{\partial y_j(n)} = -1, \\ y_j(n) &= \varphi(v_j(n)) & \Rightarrow & \frac{\partial y_j(n)}{\partial v_j(n)} = \varphi'(v_j(n)), \\ v_j(n) &= \sum_{i=0}^p w_{ji}(n) y_i(n) & \Rightarrow & \frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n). \end{aligned}$$

Therefore

$$\begin{aligned}\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} &= -e_j(n) \varphi'(v_j(n)) y_i(n), \\ \Delta w_{ji}(n) &= -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \eta e_j(n) \varphi'(v_j(n)) y_i(n) = \eta \delta_j(n) y_i(n),\end{aligned}$$

where the local gradient is

$$\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} = e_j(n) \varphi'(v_j(n)).$$

(b) Hidden neuron (4 points). A hidden neuron has no desired response $d_j(n)$, so $e_j(n)$ is not defined and the result of (a) cannot be used directly. The error must be computed recursively from the neurons k that neuron j feeds. The definition of the local gradient still applies, and $\frac{\partial v_j}{\partial w_{ji}} = y_i$ as before, so $\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$ with

$$\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \varphi'(v_j(n)).$$

With $\mathcal{E}(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n)$, and since y_j affects e_k only through v_k ,

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial v_k(n)} \cdot \frac{\partial v_k(n)}{\partial y_j(n)}.$$

From $e_k(n) = d_k(n) - \varphi(v_k(n))$ and $v_k(n) = \sum_{j=0}^q w_{kj}(n) y_j(n)$:

$$\frac{\partial e_k(n)}{\partial v_k(n)} = -\varphi'(v_k(n)), \quad \frac{\partial v_k(n)}{\partial y_j(n)} = w_{kj}(n).$$

Hence

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = -\sum_k e_k(n) \varphi'(v_k(n)) w_{kj}(n) = -\sum_k \delta_k(n) w_{kj}(n),$$

and

$$\boxed{\delta_j(n) = \varphi'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n)}$$

(c) Logistic activation (2 points).

$$\varphi(v) = \frac{1}{1 + e^{-av}} \implies \varphi'(v) = \frac{a e^{-av}}{(1 + e^{-av})^2} = a \cdot \frac{1}{1 + e^{-av}} \cdot \frac{e^{-av}}{1 + e^{-av}} = a \varphi(v) [1 - \varphi(v)],$$

using $1 - \varphi(v) = \frac{e^{-av}}{1 + e^{-av}}$. With $y_j = \varphi(v_j)$, and writing o_j for the output of an output neuron:

$$\begin{aligned}\text{output neuron: } \delta_j(n) &= a [d_j(n) - o_j(n)] o_j(n) [1 - o_j(n)], \\ \text{hidden neuron: } \delta_j(n) &= a y_j(n) [1 - y_j(n)] \sum_k \delta_k(n) w_{kj}(n).\end{aligned}$$

Marking: (a) chain rule written, 1 pt; four factors, 2 pts; Δw_{ji} and δ_j , 1 pt. (b) reason, 1 pt; $\delta_j = -\frac{\partial \mathcal{E}}{\partial y_j} \varphi'(v_j)$, 1 pt; chain through v_k with both factors, 1 pt; final δ_j with the sum over k , 1 pt. (c) φ' , 1 pt; two final formulas, 1 pt.

Part IV. Calculation

Problem 1.

(a) True costs to G: $h^*(D) = 2$ (D-G), $h^*(C) = 3$ (C-D-G), $h^*(B) = 6$ (B-C-D-G), $h^*(A) = 7$ (A-B-C-D-G), $h^*(S) = 9$ (S-A-B-C-D-G).

Node	S	A	B	C	D	G
h	8	7	6	2	2	0
h^*	9	7	6	3	2	0

$h(n) \leq h^*(n)$ for every node, so h is admissible.

(b) Not consistent. Edge B-C: $h(B) = 6 > c(B, C) + h(C) = 3 + 2 = 5$.

(c) A* trace:

Step	Expanded	g	f	Successors added ($f = g + h$)
1	S	0	8	A: $2 + 7 = 9$; B: $4 + 6 = 10$
2	A	2	9	B via A: $3 + 6 = 9$; C: $7 + 2 = 9$
3	B (via A)	3	9	C: $6 + 2 = 8$; D: $9 + 2 = 11$
4	C (via B)	6	8	D: $7 + 2 = 9$; G: $10 + 0 = 10$
5	D	7	9	G: $9 + 0 = 9$
6	G	9	9	Goal

Solution: S-A-B-C-D-G, cost 9. Note that f drops from 9 at B to 8 at C, which is the visible effect of the inconsistency in (b). The result is still optimal because h is admissible. G was generated with $f = 10$ at step 4 but not selected, since D had a lower f . The copy of C reached directly from A ($g = 7$, $f = 9$) is discarded because C was already expanded with a lower g ; in a pure tree search it would be expanded before D, without changing the result.

(d) Greedy: from S choose B ($h = 6 < 7$). From B choose C ($h = 2$, tie with D broken alphabetically). From C choose G ($h = 0$). Path S-B-C-G, cost $4 + 3 + 4 = 11$. Not optimal.

Problem 2.

(a) $D = \max(3, 5) = 5$, $E = \max(6, 9) = 9$, $F = \max(1, 2) = 2$, $G = \max(0, -1) = 0$.

$B = \min(5, 9) = 5$, $C = \min(2, 0) = 0$, $A = \max(5, 0) = 5$.

(b)

Step	Action	Bounds and result
1	Evaluate leaves 3 and 5	$D = 5$. At B: $\beta = 5$
2	Evaluate leaf 6 at E	$v = 6 \geq \beta = 5$. Prune leaf 9. $B = 5$. At A: $\alpha = 5$
3	Evaluate leaves 1 and 2 at F	$F = 2$
4	At C	$v = 2 \leq \alpha = 5$. Prune node G (leaves 0 and -1)
5	Root	$A = 5$; MAX moves to B

Evaluated: 3, 5, 6, 1, 2. Pruned: 9, 0, -1. Leaf 2 cannot be pruned: F is a MAX node, and a MAX node prunes only when $v \geq \beta$, with $\beta = +\infty$ there.

Problem 3.

Epoch	Sample	$\mathbf{w}^\top \mathbf{x}$	Action	$\mathbf{w}$ after
1	(2, 2), ω_1	$2 - 2 + 0 = 0$	≤ 0 : add $0.5 \mathbf{x}$	[2, 0, 0.5]
1	(1, -1), ω_2	$2 + 0 + 0.5 = 2.5$	≥ 0 : subtract $0.5 \mathbf{x}$	[1.5, 0.5, 0]
1	(-1, 1), ω_2	$-1.5 + 0.5 = -1$	none	[1.5, 0.5, 0]
1	(3, 1), ω_1	$4.5 + 0.5 = 5$	none	[1.5, 0.5, 0]
2	(2, 2), ω_1	4	none	[1.5, 0.5, 0]
2	(1, -1), ω_2	$1.5 - 0.5 = 1$	≥ 0 : subtract $0.5 \mathbf{x}$	[1, 1, -0.5]
2	(-1, 1), ω_2	-0.5	none	[1, 1, -0.5]
2	(3, 1), ω_1	3.5	none	[1, 1, -0.5]
3	all four	3.5, -0.5, -0.5, 3.5	none	converged

Final $\mathbf{w} = [1, 1, -0.5]^\top$. Decision line: $x_1 + x_2 - 0.5 = 0$. Three updates in total.

Problem 4. Total = 200.

$$\begin{aligned}
 \text{Accuracy} &= \frac{TP + TN}{TP + FP + TN + FN} = \frac{40 + 130}{200} = 0.8500 \\
 \text{Precision} &= \frac{TP}{TP + FP} = \frac{40}{60} = 0.6667 \\
 \text{Recall} &= \frac{TP}{TP + FN} = \frac{40}{50} = 0.8000 \\
 \text{Specificity} &= \frac{TN}{TN + FP} = \frac{130}{150} = 0.8667 \\
 F_1 &= \frac{2PR}{P + R} = \frac{TP}{TP + \frac{1}{2}(FP + FN)} = \frac{40}{40 + 15} = 0.7273 \\
 \text{Balanced accuracy} &= \frac{0.8000 + 0.8667}{2} = 0.8333 \\
 \text{MCC} &= \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \\
 &= \frac{5200 - 200}{\sqrt{60 \cdot 50 \cdot 150 \cdot 140}} = \frac{5000}{7937.25} = 0.6299
 \end{aligned}$$

Problem 5.

(a) Forward pass:

$$\begin{aligned}
 v_{h_1} &= 0 + 0.5(1) - 0.5(1) = 0, & y_{h_1} &= \varphi(0) = 0.5000 \\
 v_{h_2} &= 0.2 + 0.3(1) + 0.1(1) = 0.6, & y_{h_2} &= \varphi(0.6) = 0.6457 \\
 v_o &= -0.1 + 0.8(0.5) + 0.4(0.6457) = 0.5583, & o &= \varphi(0.5583) = 0.6361 \\
 e &= d - o = 0 - 0.6361 = -0.6361, & \mathcal{E} &= \frac{1}{2}e^2 = 0.2023
 \end{aligned}$$

(b) Local gradients:

$$\begin{aligned}
 \delta_o &= e o (1 - o) = (-0.6361)(0.6361)(0.3639) = -0.1472 \\
 \delta_{h_1} &= y_{h_1} (1 - y_{h_1}) \delta_o w_{o, h_1} = (0.5)(0.5)(-0.1472)(0.8) = -0.0294 \\
 \delta_{h_2} &= y_{h_2} (1 - y_{h_2}) \delta_o w_{o, h_2} = (0.6457)(0.3543)(-0.1472)(0.4) = -0.0135
 \end{aligned}$$

(c) Updates, $\Delta w = \eta \delta y_{\text{input}}$ with $\eta = 1$:

Weight	Δw	New value
Output bias	$(-0.1472)(1) = -0.1472$	-0.2472
Output from h_1	$(-0.1472)(0.5) = -0.0736$	0.7264
Output from h_2	$(-0.1472)(0.6457) = -0.0951$	0.3049
h_1 bias	-0.0294	-0.0294
h_1 from x_1	-0.0294	0.4706
h_1 from x_2	-0.0294	-0.5294
h_2 bias	-0.0135	0.1865
h_2 from x_1	-0.0135	0.2865
h_2 from x_2	-0.0135	0.0865

Check: the new forward pass gives $o = 0.5730$ and $\mathcal{E} = 0.1641$, down from 0.2023.

Problem 6.

(a) $\|\mathbf{w}\| = \sqrt{3^2 + 4^2} = 5$. Distance from origin $= \frac{|w_0|}{\|\mathbf{w}\|} = \frac{12}{5} = 2.4$. $g(2, 1) = 6 + 4 - 12 = -2$, so the distance is $\frac{|g(\mathbf{x})|}{\|\mathbf{w}\|} = \frac{2}{5} = 0.4$ and the point is on the negative side (assigned to ω_2).

(b) $\frac{dJ}{d\theta} = 2(\theta - 3)$ and $\theta(k+1) = \theta(k) - \mu \cdot 2(\theta(k) - 3)$:

$$\theta(1) = 0 - 0.25 \cdot 2 \cdot (-3) = 1.5,$$

$$\theta(2) = 1.5 - 0.25 \cdot 2 \cdot (-1.5) = 2.25,$$

$$\theta(3) = 2.25 - 0.25 \cdot 2 \cdot (-0.75) = 2.625.$$

Here $R = 2$, so $\lambda_{\max} = 2$ and the iteration converges for $0 < \mu < \frac{2}{\lambda_{\max}} = 1$.

Part V. Essay: points a full answer should contain

Essay 1.

- Simple reflex: current percept only, condition-action rules; efficient but limited.
- Model-based reflex: internal state, transition model, and sensor model; handles partial observability.
- Goal-based: considers the future and plans action sequences (search); more flexible, less efficient.
- Utility-based: utility function maps states to real numbers and gives the trade-off between conflicting goals.
- Taxi environment: partially observable, multi-agent, stochastic, sequential, dynamic, continuous.
- Conclusion: a model-based utility-based agent, usually with learning. Partial observability requires a model; reaching a destination requires goals; speed vs. safety vs. fare requires utilities. A reflex or table-driven agent cannot cover this environment.

Essay 2.

- Law 4: when knowledge is not yet acquired or codified, trial-and-error search can still reach expert performance. Example: Deep Blue, with a small knowledge base and 200 million moves per second.
- Law 5: knowledge reduces uncertainty and constrains the exponential growth of search. Examples: Rubik's cube with practice; Sphinx error rate 4% to 30% when syntactic knowledge is removed.
- Link to A*: $h(n)$ is the knowledge. With $h = 0$ (no knowledge) A* is uniform-cost search and expands the most nodes. A more accurate admissible heuristic (one that dominates) lowers the effective branching factor toward 1.
- Trade-off: a very expensive heuristic can cost more than the search it saves (base-level/meta-level trade-off). Bounded rationality motivates settling for good-enough solutions.

Essay 3.

- XOR is not linearly separable; a single perceptron draws one hyperplane. Minsky and Papert (1969) proved the limitation, which contributed to reduced funding and the first AI winter (1970s dormancy).
- A two-layer perceptron solves it: the hidden layer maps inputs to a feature space where the classes are linearly separable; the output neuron separates them.
- Training hidden layers needed a way to assign error to hidden neurons. Backpropagation (Rumelhart, Hinton, Williams, 1986) does this with the chain rule: δ for hidden neurons is computed recursively from the next layer. It requires a differentiable nonlinear activation.
- Limitations: convergence to local minima only; slow training and sensitivity to η (addressed by momentum); overfitting and the need for enough training data ($N \geq W/\epsilon$); learned weights are hard to interpret.

Practice Exam 2: Answer Key

Part I. Identification

1. Laws of Thought Approach (thinking rationally)
2. Herbert Simon, 1957
3. $70,000 \pm 20,000$
4. Any-time algorithm
5. Utility (utility function)
6. Autonomy
7. Exploratory problem
8. Uniform-cost search
9. Admissible heuristic
10. Iterative Deepening A* (IDA*)
11. Effective branching factor
12. Alpha (α)
13. Expectiminimax
14. Quiescent positions
15. AC-3
16. Cutset conditioning
17. Adaline (adaptive linear element)
18. Correlation matrix
19. Specificity (true negative rate)
20. Recurrent network

Part II. True or False

1. TRUE.
2. TRUE. Together with the ALPAC Report in the US.
3. FALSE. It uses only the current percept. The table-driven agent indexes on the whole percept sequence.
4. FALSE. It is semi-dynamic: the environment does not change but the performance score does.
5. TRUE.
6. TRUE. This holds when every operator has a non-negative cost.
7. TRUE.
8. TRUE.
9. FALSE. Memory space, more than computation time, is A*'s main drawback.
10. TRUE.
11. TRUE.
12. FALSE. It returns the move whose node has the highest number of playouts.
13. FALSE. The evaluation must be a positive linear transformation of the probability of winning; magnitudes matter at chance nodes.
14. FALSE. Values are deleted from X , the tail.
15. FALSE. It is depth-first search with single-variable assignments.
16. TRUE.
17. FALSE. Low bias and high variance.
18. TRUE. That is why they should not be used to compare classifiers across data sets.
19. TRUE.

20. FALSE. It increases in magnitude (acceleration). It decreases when the sign alternates.

Part III. Proving and Derivation

1. Consistency (5 points).

(a) *Consistent* $\Rightarrow$ *admissible*. Induction on m , the number of steps on a cheapest path from n to a goal. Base, $m = 0$: n is a goal, so $h(n) = 0 = h^*(n)$.

Step: assume $h(n') \leq h^*(n')$ for all nodes whose cheapest path has m steps. Let n have a cheapest path of $m + 1$ steps, with first successor n' on that path. Then

$$h(n) \leq c(n, n') + h(n') \leq c(n, n') + h^*(n') = h^*(n).$$

The first inequality is consistency, the second is the induction hypothesis, and the equality holds because n' lies on a cheapest path from n . ■

(b) f is non-decreasing. For a successor n' of n ,

$$f(n') = g(n') + h(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n). \quad \blacksquare$$

2. XOR (5 points).

(a) Suppose weights w_1, w_2 and threshold weight w_0 exist such that the output is 1 when $w_1x_1 + w_2x_2 + w_0 \geq 0$ and 0 otherwise. The four input patterns require

$$(0, 0) \mapsto 0 : w_0 < 0 \quad (i)$$

$$(1, 0) \mapsto 1 : w_1 + w_0 \geq 0 \quad (ii)$$

$$(0, 1) \mapsto 1 : w_2 + w_0 \geq 0 \quad (iii)$$

$$(1, 1) \mapsto 0 : w_1 + w_2 + w_0 < 0 \quad (iv)$$

Adding (ii) and (iii): $w_1 + w_2 + 2w_0 \geq 0$, so $w_1 + w_2 + w_0 \geq -w_0 > 0$ by (i). This contradicts (iv). No such weights exist. ■

(b) With the step function $f(v) = 1$ if $v \geq 0$, else 0:

$$y_1 = f\left(x_1 + x_2 - \frac{1}{2}\right), \quad y_2 = f\left(x_1 + x_2 - \frac{3}{2}\right), \quad \text{output} = f\left(y_1 - 2y_2 - \frac{1}{2}\right).$$

x_1	x_2	y_1	y_2	$y_1 - 2y_2 - \frac{1}{2}$	Output
0	0	0	0	$-\frac{1}{2}$	0
0	1	1	0	$\frac{1}{2}$	1
1	0	1	0	$\frac{1}{2}$	1
1	1	1	1	$-\frac{3}{2}$	0

3. Full derivation with the hyperbolic tangent activation (10 points).

(a) $\Delta w_{ji} = \eta \delta_j y_i$ for every neuron (2 points). For any neuron j , the weight w_{ji} affects $\mathcal{E}$ only through v_j . By the chain rule and the definition of δ_j ,

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)} = -\delta_j(n) y_i(n),$$

because $v_j(n) = \sum_{i=0}^p w_{ji}(n) y_i(n)$ gives $\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$. Gradient descent then gives

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \eta \delta_j(n) y_i(n).$$

Nothing in this step depends on whether j is an output or a hidden neuron.

(b) The local gradient (4 points).

Output neuron. v_j affects $\mathcal{E}$ through y_j and then e_j :

$$\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} = -e_j(n) \cdot (-1) \cdot \varphi'(v_j(n)) = e_j(n) \varphi'(v_j(n)).$$

Hidden neuron. There is no desired response, so e_j does not exist. Instead

$$\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \varphi'(v_j(n)).$$

y_j affects $\mathcal{E} = \frac{1}{2} \sum_k e_k^2$ only through the v_k of the neurons it feeds:

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial v_k(n)} \cdot \frac{\partial v_k(n)}{\partial y_j(n)} = \sum_k e_k(n) [-\varphi'(v_k(n))] w_{kj}(n) = -\sum_k \delta_k(n) w_{kj}(n),$$

using $e_k = d_k - \varphi(v_k)$, $v_k = \sum_j w_{kj} y_j$, and $\delta_k = e_k \varphi'(v_k)$. Therefore

$$\delta_j(n) = \varphi'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n).$$

(c) Hyperbolic tangent (2 points).

$$\begin{aligned} \varphi(v) = a \tanh(bv) &\implies \varphi'(v) = ab \operatorname{sech}^2(bv) = ab[1 - \tanh^2(bv)] = ab \left[1 - \frac{\varphi^2(v)}{a^2}\right] \\ &= \frac{b}{a} [a^2 - \varphi^2(v)] = \frac{b}{a} [a - \varphi(v)][a + \varphi(v)]. \end{aligned}$$

Hence

$$\begin{aligned} \text{output neuron: } \delta_j(n) &= \frac{b}{a} [d_j(n) - o_j(n)] [a^2 - o_j^2(n)], \\ \text{hidden neuron: } \delta_j(n) &= \frac{b}{a} [a^2 - y_j^2(n)] \sum_k \delta_k(n) w_{kj}(n). \end{aligned}$$

(d) Generalized delta rule (2 points).

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n), \quad \alpha > 0.$$

(i) Same sign on consecutive iterations: the terms add, $\Delta w_{ji}(n)$ grows in magnitude, and descent accelerates in the downhill direction. (ii) Opposite signs: the terms partly cancel, $\Delta w_{ji}(n)$ shrinks, and the weight is adjusted by a small amount, which damps oscillation and stabilizes the network. Momentum therefore speeds up learning without the instability of a large η .

Part IV. Calculation

Problem 1.

Strategy	Expansion order	Goal	Path	Cost
BFS	S, A, B, D, E, F, G_1	G_1	S-B- G_1	9
DFS	S, A, D, E, G_2	G_2	S-A-E- G_2	5
UCS	S (0), B (1), A (2), E (3), G_2 (5)	G_2	S-A-E- G_2	5
IDS	Limit 0: S. Limit 1: S, A, B. Limit 2: S, A, D, E, B, F, G_1	G_1	S-B- G_1	9

- UCS frontier after expanding E: G_2 (5), D (6), F (6), G_1 (9). G_2 is selected first.
- The optimal cost is 5. UCS is guaranteed to find it because all step costs are positive. DFS found it only because of the left-to-right ordering; it has no guarantee.
- BFS and IDS return the shallowest goal (depth 2), which is not the cheapest here because step costs are not identical.

Problem 2.

(a) MIN values: $\min(5, 7, 3) = 3$, $\min(4, 1, 8) = 1$, $\min(6, 2, 9) = 2$. Root = $\max(3, 1, 2) = 3$.

Alpha-beta: the first MIN node is fully evaluated (value 3, so $\alpha = 3$). Second MIN node: leaf 4 ($v = 4 > 3$, continue), leaf 1 ($v = 1 \leq \alpha = 3$), prune leaf 8. Third MIN node: leaf 6 (continue), leaf 2 ($v = 2 \leq 3$), prune leaf 9. Seven leaves evaluated; 8 and 9 pruned.

(b)

$$a_1 : 0.5 \cdot \min(6, 10) + 0.5 \cdot \min(2, 12) = 0.5(6) + 0.5(2) = 4.0$$

$$a_2 : 0.25 \cdot \min(20, 8) + 0.75 \cdot \min(4, 5) = 0.25(8) + 0.75(4) = 5.0$$

MAX chooses a_2 .

Problem 3.

Stage	A	B	C	D	E
Start	$\{R, G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$
(a) $A = R$	R	$\{G, B\}$	$\{G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$
(b) $B = G$	R	G	$\{B\}$	$\{R, B\}$	$\{R, G, B\}$
(c) Arc consistency	R	G	$\{B\}$	$\{R\}$	$\{G, B\}$

(c) Deletions:

1. Arc $D \rightarrow C$: $D = B$ has no allowed value in $C = \{B\}$. Delete B from D , so $D = \{R\}$.
2. D lost a value, so recheck its neighbors. Arc $E \rightarrow D$: $E = R$ has no allowed value in $D = \{R\}$. Delete R from E , so $E = \{G, B\}$.
3. All other arcs are consistent. No domain is empty, so no failure is detected.

(d) Not a tree: $A-B-C$ and $B-C-D$ are cycles. Removing B (or C) leaves the chain $A-C-D-E$ (or $A-B-D-E$), which is a tree. Cutset size $c = 1$. Runtime bound

$$O(d^c (n - c) d^2) = 3 \cdot 4 \cdot 9 = 108 \text{ steps, against } d^n = 3^5 = 243 \text{ assignments for brute force.}$$

Problem 4. Total = 200.

$$\text{Accuracy} = \frac{30 + 40 + 90}{200} = 0.8000$$

Class	Recall = $\frac{TP}{\text{row total}}$	Precision = $\frac{TP}{\text{column total}}$
X	30/40 = 0.7500	30/40 = 0.7500
Y	40/60 = 0.6667	40/55 = 0.7273
Z	90/100 = 0.9000	90/105 = 0.8571

Class Y: $TP = 40$; $FN = 10 + 10 = 20$; $FP = 5 + 10 = 15$; $TN = 200 - 40 - 20 - 15 = 125$.

$$\text{Balanced accuracy} = \frac{0.7500 + 0.6667 + 0.9000}{3} = 0.7722$$

Problem 5.

k	$\mathbf{x}_k$ (extended)	y_k	$\mathbf{x}_k^T \mathbf{w}(k-1)$	Error	$\mathbf{w}(k)$
1	[1, 2, 1]	+1	0	1.0	[0.1, 0.2, 0.1]
2	[2, -1, 1]	-1	$0.2 - 0.2 + 0.1 = 0.1$	-1.1	[-0.12, 0.31, -0.01]
3	[0, 1, 1]	+1	$0.31 - 0.01 = 0.30$	0.7	[-0.12, 0.38, 0.06]

Difference: LMS updates on every sample by an amount proportional to the error of the linear output (no activation function), and converges asymptotically to the MSE solution even when the classes are not separable. The perceptron updates only when a sample is misclassified and converges only for linearly separable classes.

Problem 6.

(a) Forward pass:

$$\begin{aligned}
 v_{h_1} &= 0.1 + 0.2(0.5) + 0.4(1) = 0.6, & y_{h_1} &= 0.6457 \\
 v_{h_2} &= -0.2 + 0.6(0.5) - 0.1(1) = 0, & y_{h_2} &= 0.5000 \\
 v_{o_1} &= 0.3 + 0.5(0.6457) - 0.6(0.5) = 0.3228, & o_1 &= 0.5800 \\
 v_{o_2} &= -0.1 + 0.7(0.6457) + 0.2(0.5) = 0.4520, & o_2 &= 0.6111 \\
 e_1 &= 1 - 0.5800 = 0.4200, & e_2 &= 0 - 0.6111 = -0.6111
 \end{aligned}$$

$$\mathcal{E} = \frac{1}{2}(e_1^2 + e_2^2) = \frac{1}{2}(0.1764 + 0.3734) = 0.2749$$

(b) Local gradients:

$$\begin{aligned}
 \delta_{o_1} &= e_1 o_1 (1 - o_1) = (0.4200)(0.5800)(0.4200) = 0.1023 \\
 \delta_{o_2} &= e_2 o_2 (1 - o_2) = (-0.6111)(0.6111)(0.3889) = -0.1452 \\
 \delta_{h_1} &= y_{h_1} (1 - y_{h_1}) [\delta_{o_1} w_{o_1, h_1} + \delta_{o_2} w_{o_2, h_1}] = (0.6457)(0.3543) [(0.1023)(0.5) + (-0.1452)(0.7)] \\
 &= (0.2288)(-0.0505) = -0.0116 \\
 \delta_{h_2} &= y_{h_2} (1 - y_{h_2}) [\delta_{o_1} w_{o_1, h_2} + \delta_{o_2} w_{o_2, h_2}] = (0.25) [(0.1023)(-0.6) + (-0.1452)(0.2)] \\
 &= (0.25)(-0.0904) = -0.0226
 \end{aligned}$$

(c) Updates with $\eta = 0.5$:

Weight	$\Delta w = \eta \delta y_{\text{input}}$	New value
o_1 bias	$0.5(0.1023)(1) = 0.0512$	0.3512
o_1 from h_1	$0.5(0.1023)(0.6457) = 0.0330$	0.5330
o_1 from h_2	$0.5(0.1023)(0.5) = 0.0256$	-0.5744
h_1 bias	$0.5(-0.0116)(1) = -0.0058$	0.0942
h_1 from x_1	$0.5(-0.0116)(0.5) = -0.0029$	0.1971
h_1 from x_2	$0.5(-0.0116)(1) = -0.0058$	0.3942

(d)

$$\Delta w(n) = \alpha \Delta w(n-1) + \eta \delta_{o_1} y_{h_1} = 0.9(0.02) + 0.0330 = 0.0510.$$

The previous change had the same sign, so momentum enlarges the step.

Part V. Essay: points a full answer should contain

Essay 1.

	BFS	DFS	IDS	A*
Complete	Yes (b finite)	No	Yes	Yes (locally finite graph, costs $\geq \delta > 0$)
Optimal	If step costs identical	No	If step costs identical	Yes, with admissible h
Time	$O(b^d)$	$O(b^m)$	$O(b^d)$	Exponential unless h is very accurate
Space	$O(b^d)$	$O(bm)$	$O(bd)$	Exponential; the main drawback

- With no heuristic, A* reduces to uniform-cost search and offers no advantage in guidance.
- Choice: iterative deepening. It combines the completeness and shallowest-solution property of BFS with the linear space of DFS, and the repeated work is small (about 11% for $b = 10$).
- DFS risks never terminating on deep or infinite trees; BFS runs out of memory.

Essay 2.

- Accuracy is misleading on unbalanced data. If 5% of patients have the disease, a classifier that labels everyone healthy is 95% accurate and detects no one.
- Example consistent with 97%: $TP = 30$, $FN = 20$, $FP = 10$, $TN = 940$. Accuracy is 0.97, but sensitivity is $30/50 = 0.60$: 40% of diseased patients are missed.
- Ask for the confusion matrix, sensitivity (recall), specificity, precision/PPV, NPV, F_1 or F_β ($\beta = 2$ if missing a case is costlier), balanced accuracy, MCC, and the ROC curve with AUC.
- Costs differ: a false negative (missed disease) is usually worse than a false positive (false alarm).
- Threshold: lowering it classifies more cases as diseased, raising sensitivity and lowering specificity; raising it does the opposite. The ROC curve shows the whole trade-off and AUC summarizes it.
- PPV and NPV depend on prevalence, so they do not transfer between data sets with different class ratios.

Essay 3.

- Decomposition:

$$E_D \left[\left(g(\mathbf{x}; D) - E[y | \mathbf{x}] \right)^2 \right] = \underbrace{\left(E_D[g(\mathbf{x}; D)] - E[y | \mathbf{x}] \right)^2}_{\text{bias}^2} + \underbrace{E_D \left[\left(g(\mathbf{x}; D) - E_D[g(\mathbf{x}; D)] \right)^2 \right]}_{\text{variance}}$$

Bias is how far the average model over data sets is from $E[y | \mathbf{x}]$; variance is the sensitivity to the particular data set.

- Too many parameters relative to N : low bias, high variance. The network memorizes the training data (overfitting); the learned mapping is not smooth and generalization is poor.
- Too simple a model: high bias, low variance.
- Generalization depends on training set size, network architecture, and problem complexity. Rule of thumb $N \geq W/\epsilon$ (about 10 times the number of weights for 10% error).
- Practices from the lecture, any three: stop training when generalization performance peaks; normalize inputs (mean removal, decorrelation, covariance equalization); initialize weights in $[-2.4/F_i, +2.4/F_i]$; randomize the order of examples each epoch; use tanh with $a = 1.716$, $b = 2/3$; add momentum; offset target values from the limiting values; scale the learning rate by fan-in.